A 30-Minute Talk on Early-Exit Speculative Decoding for Hindi

Narration Script, Design Justifications, and 50 Q&A Pairs

Team Transformers
KSPSVLN Siddardha Kumar Kavuri (2025201061)
Piyush Kumar Agarwal (2025201082)
Sankalp Chaturvedi (2025201083)
Aviral Tyagi (2025201086)

April 2026

Abstract

This document is a complete speaker's guide for a 30-minute academic presentation on our project *Early-Exit Speculative Decoding for Hindi*. It is divided into three parts: **Part A** is the narration, written in plain language with stories, analogies, and embedded figures; **Part B** is a set of short, professor-ready justifications for every major design choice; **Part C** is a bank of 50 question-and-answer pairs for examination preparation. A safe speaking pace is 130–140 words per minute, so the narration in Part A targets roughly 4,000 words over 30 minutes. Each numbered section in Part A is marked with the specific figure or table to display while you speak.

Contents

Part I
The Talk — Narration for 30 Minutes

1 Opening (~2 minutes)

> **Slide:** Title slide — "Early-Exit Speculative Decoding for Hindi", Team Transformers, names, roll numbers.

Good morning everyone. I am presenting our final project for this course, done together with my teammates Siddardha, Sankalp, and Aviral. We call ourselves Team Transformers.

In one sentence: **our project makes large language models generate Hindi text faster, without training a second model, and without changing the text they produce.**

The way we do this is to attach a few small "exit heads" to the middle layers of a single language model. Those heads quickly *guess* the next few tokens. The full model then *verifies* the guesses in one batched forward pass. The tokens that match are kept; the rest are thrown away.

Over the next 30 minutes, I will tell this story in six chapters:

1. Why generating text with an LLM is slow.
2. Two older ideas — *speculative decoding* and *early exiting* — and how we combined them.
3. How we trained the exit heads, using the ideas of *teacher forcing* and *knowledge distillation*.
4. The 12 different decoding strategies we compared.
5. The numbers — acceptance rate, speedup, throughput.
6. A closer look at Hindi morphology and what it teaches us.

2 The Problem: One Token at a Time (~2 minutes)

> **Slide:** A cartoon of a transformer stack producing one word, with the phrase "do it again, 100 times" next to it.

Let me first explain *why* LLM inference is slow. Modern language models — GPT-4, LLaMA, Qwen — all generate text one token at a time. To produce the next word, the model must look at every token it has produced so far, and run a full forward pass through *every* transformer layer.

Our base model, **Qwen2-1.5B**, has 28 layers. Suppose we want to generate 100 tokens of Hindi text. That is:

$$28 \text{ layers} \times 100 \text{ tokens} = 2{,}800 \text{ layer evaluations.}$$

> **Analogy.** Think of it like writing a sentence by asking 28 experts in a row for their opinion on the *next word*, and then repeating that process 100 times. Each word depends on all previous words, so you cannot ask the experts about word 5 before word 4 is finalised.

This is what we call the *sequential bottleneck*: you cannot parallelise across the token dimension. The whole engineering question of this project is: **can we cheat this bottleneck, without changing the final output?**

3 Idea 1 — Speculative Decoding (~3 minutes)

Slide: A little "draft model" on the left whispers 3 candidate tokens into a big "target model" on the right; the target does one pass and marks each as ✓or ×.

The first idea, from Leviathan et al. and Chen et al. in 2023, is called **speculative decoding**. It works like this:

1. Take a small, fast model — the *draft model* — and let it quickly guess the next K tokens.
2. Put those K guesses into the big *target model*, all together, in one batched forward pass.
3. The target model tells you, for each of the K positions, what *it* would have sampled.
4. You accept the draft tokens greedily from left to right until the first mismatch.

If the draft's first three guesses agree with the target, congratulations — in the cost of *one* forward pass of the big model, you produced *three* tokens instead of one. That is a 3× speedup on that step.

Analogy. Imagine a fast junior typist and a careful senior editor. The junior types three words at a time. The editor skims all three in one glance. If all three match what the editor would have written, great, keep them. At the first wrong word, the editor rewrites from there.

The cost: you now need two models. Hosting a small draft model is extra memory and extra engineering — the two models must share a tokeniser, be kept in sync, etc.

So the real question becomes: can we do speculative decoding *without a second model*?

4 Idea 2 — Early Exiting (~2 minutes)

Slide: A tall transformer with three small "exit classifiers" dangling off layers 8, 16, and 22.

The second idea, from BranchyNet in 2016 and later DeeBERT and CALM in NLP, is called **early exiting**. The idea is: attach a small classifier at an intermediate layer of the network. For easy inputs, that classifier is already confident enough — so you stop computation and output immediately, without running the rest of the layers.

Analogy. When you read a multiple-choice question, sometimes the first three words already make the answer obvious, and you do not need to read the full sentence. Early exiting is the same idea for a neural network.

By itself, early exiting makes each token-generation cheaper, but does not remove the sequential bottleneck — you still generate one token at a time.

5 Our Approach — EESD = Speculative Decoding + Early Exit (~2 minutes)

Slide: The Qwen2 stack with exit heads at layers 8, 16, 22 marked *draft*; layers 23–28 marked *verify*.

Liu et al. (2024) combined these two ideas into **Early-Exit Speculative Decoding**, or EESD. The same big model plays both roles:

- Layers 1 through d, plus a tiny exit head, act as the *draft*. They quickly produce K guesses.
- Then all 28 layers verify those guesses in one batched pass.

Two beautiful consequences:

1. **No second model.** The exit heads add less than 0.5% extra parameters.
2. **Better distribution alignment.** Because the draft literally comes from the same model's own hidden states, its guesses naturally look like the target model's guesses.

Our project takes this framework and pushes it deeper: we compare 12 decoding variants, run a morphological analysis on Hindi, and test robustness on 500 prompts.

6 Why Hindi? (~2 minutes)

Slide: Hindi verb inflection chart: करता, करती, करते.

We chose Hindi deliberately, because it is *morphologically rich*. A single verb like करना ("to do") inflects many ways depending on the subject:

- करता — masculine, singular, habitual
- करती — feminine, singular, habitual
- करते — masculine, plural, or formal

These forms differ only in the last character, but semantically carry gender, number, and formality. For a *shallow* draft model to get these right, it must already know the subject of the sentence. That makes Hindi a genuinely hard test-bed.

If EESD works on Hindi, we can reasonably hope it works on other Indian languages (Tamil, Bengali, Telugu) and agglutinative languages like Turkish or Finnish.

7 Architecture: Two Kinds of Exit Heads (~3 minutes)

Slide: Two box diagrams side by side — Hook head and Bottleneck head.

We attach exit heads at three depths: **layer 8 (shallow), 16 (middle), and 22 (deep)**. Layer 28 is the full model output. We chose three depths, spaced roughly evenly across the 28-layer stack, to capture a spectrum from "cheap but inaccurate" to "almost full and almost as accurate".

We compare two head architectures:

Hook head. The simplest possible design:

$$\text{LayerNorm} \;\rightarrow\; \text{Linear}(1536 \rightarrow 151{,}936)$$

We call it "hook" because it hooks into the hidden state and nothing else.

Bottleneck head (BN). A thin, two-stage head with a narrow middle:

$$\text{LayerNorm} \;\rightarrow\; \text{Linear}(1536 \rightarrow 256) \;\rightarrow\; \text{activation} \;\rightarrow\; \text{Linear}(256 \rightarrow 151{,}936)$$

Analogy for the bottleneck. Imagine summarising a 500-page novel into a single paragraph, and then recreating the plot from that paragraph. The compression into one paragraph forces you to keep only the essential facts. That is what the 256-dim bottleneck does for our exit head — it forces compression, which acts as a regulariser and prevents the head from overfitting to the hidden state.

We will see in the training diagnostics that BN consistently beats Hook on training loss, validation loss, and acceptance rate.

8 Training: Teacher Forcing and Knowledge Distillation (~3 minutes)

Slide: The distillation loss equation, plus an illustration of the teacher (full model) softly labelling tokens for the student (exit head).

Now let me explain two ideas that are central to how we trained the exit heads: *teacher forcing* and *knowledge distillation*.

Teacher forcing (why and how)

During training we do *not* let the exit head generate tokens freely. Instead, for every position in a real Hindi sentence, we feed the **ground-truth previous tokens** into the base model, read out the hidden state at layer d, and train the head to predict the next token.

That practice of always feeding the correct previous tokens is called **teacher forcing**. It is the standard way to train sequence models.

Analogy. When teaching a child to recite a poem, you let them repeat each correct line even if they made a mistake one line earlier — otherwise early mistakes cascade and the child never finishes a line. Teacher forcing is the neural-network version of that.

Why does this matter here? If we instead let the exit head generate freely during training, its early mistakes would compound into wrong context, and every downstream gradient would be computed on an invalid example. Teacher forcing keeps training clean. The cost — known as *exposure bias* — is that the student never sees its own mistakes during training. For us, this is not a serious problem, because at inference time the exit head is always conditioned on tokens that came from the *full* model's own forward pass, not on its own free generations.

Knowledge distillation (why and how)

We do not train the exit heads to predict the hard ground-truth token with cross-entropy loss. Instead, we train them to imitate the *full model's own output distribution*. The full model acts as a teacher; the exit head is the student.

The loss is a temperature-scaled KL divergence (Hinton et al., 2015):

$$\mathcal{L} = T^2 \cdot D_{\mathrm{KL}}\big(\sigma(f/T) \,\|\, \sigma(\hat{f}_d/T)\big),$$

where f is the full model's logits, $\hat{f}_d$ is the exit head's logits, $T = 2$ is the temperature, and σ is softmax.

Analogy. Imagine a teacher grading a multiple-choice question. A one-hot label says only "the answer is C". A teacher using knowledge distillation says "the answer is C (80%), but B is a reasonable second choice (15%), and A is definitely wrong (1%)". The student learns much more from the richer signal. In our case, the teacher is not telling the exit head *just* which Hindi word comes next, but the whole shape of the probability distribution over the vocabulary.

Why does this matter specifically for speculative decoding? Because our goal is for the exit head's predictions to *match* the full model's predictions — that is exactly what increases acceptance rate. Soft KL is the natural loss for that.

The T^2 prefactor keeps the gradient magnitude stable as T changes; $T = 2$ is a standard choice.

Training setup

Slide: Bullet list of hyperparameters.

- **Data:** 167K Hindi sentences from AI4Bharat IndicCorp v2 (≈5M tokens).
- **Optimiser:** 8-bit AdamW via `bitsandbytes`, to cut optimiser memory ∼4×.
- **Learning rate:** 10^{-4} with cosine annealing and 5% linear warmup.
- **Effective batch size:** 8 (2 per GPU × 4 gradient accumulation).
- **Epochs:** 10.
- **Precision:** float16 base model, float32 loss computation.
- **Hardware:** H100 for training, A100 for inference.

All base model parameters are **frozen**. We only train the three exit heads.

9 The 12 Decoding Strategies (~3 minutes)

Slide: A tree of methods, grouped by baseline / fixed-depth / adaptive / weighted.

At inference time, we test 12 methods:

Baseline.

1. **AR** — plain autoregressive decoding. This is what we want to beat.

Fixed-depth (always exit at the same layer).

2. **Draft** — standard speculative decoding using the deepest exit (layer 22).
3. **Hook** — fixed depth with the Hook head.
4. **TrueExit** — same as Hook, but drafting *actually stops* the forward pass at the exit layer.
5. **BN-TrueExit** — TrueExit with the Bottleneck head.

Adaptive policies (choose the depth dynamically).

6. **Thompson** — Thompson Sampling over {8, 16, 22} with Hook heads.
7. **Thomp-BN** — Thompson Sampling with BN heads.
8. **Entropy** — pick the shallowest layer whose entropy is below a threshold.
9. **UCB-BN** — Upper Confidence Bound with BN heads, $c = 1.41$.
10. **UCB-Hook** — UCB with Hook heads.

Weighted variants (reward $= 0.92 \cdot$ **correctness** $+0.08 \cdot$ **time).**

11. **WThomp-BN** — weighted Thompson, BN heads.
12. **WThomp-Hook** — weighted Thompson, Hook heads.

Analogy for bandit policies. Imagine three restaurants, and every night you must pick one. Thompson Sampling is like keeping a private estimate of how good each restaurant is, mentally rolling a die based on each estimate, and visiting the "winner". UCB is like saying "I've only been to this one once, so I'll give it a bonus visit just to be sure". Entropy gating is the cautious version: "if I am confident about the cheap option, I'll go with it."

12 methods = 4 fixed-depth + 2 Thompson + 2 UCB + 2 weighted + Entropy + AR. We designed this grid so that every axis — head architecture, policy, reward shaping — can be ablated cleanly.

10 Decoding Strategies in Depth: How Each Works and Why We Chose Them (~6 minutes)

Slide: A 12-row table — columns: Method / Head / Policy / Reward signal / True exit? / Hyperparameters.

In the previous section I listed the 12 methods. Now let me walk through each in detail — what it does mechanically, why we included it, and which hyperparameter values we picked. The professor is most likely to probe this part, so I will go slowly.

10.1 Shared setup (applies to all drafting methods)

Three settings are the same for every speculative method:

- **Draft length $K = 3$.** From the K-ablation (Section 13). Smaller wastes verification overhead on one proposed token; larger hits diminishing returns because $\alpha(1 - \alpha^K)/(1 - \alpha)$ saturates quickly for $\alpha < 0.75$.
- **Exit layers $d \in \{8, 16, 22\}$.** Spread roughly evenly over 28 layers so we cover a shallow/middle/deep regime.
- **Greedy acceptance.** Tokens are accepted left-to-right until the first mismatch; the mismatch token is then sampled from the full target model. This is the standard speculative-decoding rule and is distribution-preserving under greedy decoding.

10.2 Baseline: AR

AR (autoregressive). Standard one-token-at-a-time decoding with a full forward pass per token. No drafting, no exits. Its only role in the study is to set the line in the sand: any speculative method worth deploying must beat AR on wall-clock time.

10.3 Fixed-depth methods (always exit at the same layer)

Draft. The closest method in our grid to "plain speculative decoding". The draft is the Qwen2 stack up through layer 22, plus the trained exit head at layer 22. We run this 22-layer drafter $K = 3$ times to produce three candidate tokens; the full 28-layer target then verifies all three in a single batched pass.

- *Hyperparameters:* $d = 22$, $K = 3$, Hook head.
- *Why depth 22 specifically?* From the per-depth heatmap, layer 22 has the highest individual α. For a fixed-depth method, the deepest exit we have is the natural choice — we want the draft to be as accurate as possible, since we pay the verification cost regardless.
- *Role in the study:* sanity-check reference point that every other method is expected to match or beat.

Hook (fixed depth). Same as Draft. We list it separately because it is our *A/B control* for the head-architecture comparison. Differences between Hook and BN-TrueExit (or Hook and TrueExit) can then be attributed cleanly to head architecture or to "true exit vs hook read-out", not to any other confound.

- *Hyperparameters:* $d = 22$, $K = 3$, Hook head.

TrueExit. Identical to Hook *except that during drafting the forward pass actually stops at layer 22.* Layers 23–28 are not executed during drafting. This directly saves wall-clock time on every draft step.

- *Hyperparameters:* $d = 22$, $K = 3$, Hook head.
- *Why it matters:* comparing TrueExit against Hook isolates the wall-clock benefit of "stopping" from the accuracy of "reading out an intermediate representation". Both use the same exit head; only the forward pass differs.

BN-TrueExit. TrueExit plus the Bottleneck head. Best fixed-depth configuration, and winner on acceptance rate (70.9%).

- *Hyperparameters:* $d = 22$, $K = 3$, Bottleneck head with hidden dim 256.
- *Why bottleneck dim 256?* Compression ratio $1536/256 = 6\times$. Small enough to force compression (regularise); large enough not to discard useful information. Empirically best in pilot runs against 128 (too aggressive) and 512 (not enough regularisation).

10.4 Adaptive policies: multi-armed bandits

Adaptive methods treat each drafting step as a *decision*: which of the three depths {8, 16, 22} should we use? Each depth is an arm of a multi-armed bandit. We pull an arm, observe a reward, and update our beliefs. The core tension is exploration (try under-used arms) vs exploitation (use the arm with highest known reward).

Thompson Sampling. Mechanically:

1. Maintain a Beta(α_d, β_d) posterior over acceptance probability at each depth d.
2. At every drafting step, sample $\theta_d \sim \text{Beta}(\alpha_d, \beta_d)$ once per depth.
3. Pick $d^\star = \arg\max_d \theta_d$ and draft at depth $d^\star$.
4. Count accepted / rejected among the K draft tokens; update $\alpha_{d^\star}$ += #accepted, $\beta_{d^\star}$ += #rejected.

- *Hyperparameters:* Beta$(1, 1)$ uniform prior. Reward is raw correctness — *no* latency weighting. Hook head.
- *Why Beta$(1, 1)$?* It is the non-informative prior — equivalent to "I have no opinion about any depth before I start". Any other prior would prejudge the bandit.
- *Why Thompson and not ε-greedy?* Principled Bayesian interpretation; exploration decays automatically as evidence accumulates; no exploration-decay schedule to tune; provably near-optimal regret for finite-arm bandits.

Thomp-BN. Identical policy to Thompson, but with Bottleneck heads. This combination — best policy × best head — is our overall speedup winner (1.130×).

- *Hyperparameters:* Beta$(1, 1)$ prior, BN head with hidden dim 256.

UCB-BN. Upper Confidence Bound (UCB1). At every step:

$$\text{UCB}_d \;=\; \hat{\mu}_d \;+\; c \cdot \sqrt{\frac{\ln N}{n_d}}, \qquad d^\star = \arg\max_d \text{UCB}_d,$$

where $\hat{\mu}_d$ is the empirical mean reward at depth d, N is the total number of draft steps so far, and n_d is the number of times we have pulled arm d.

- *Hyperparameters:*
  - Exploration constant $c = \sqrt{2} \approx 1.41$ (textbook UCB1).

  - Reward: $r = 0.92 \cdot \text{correctness} + 0.08 \cdot (1 - \tilde{t})$, where $\tilde{t}$ is normalised drafting time.
  - BN head, hidden dim 256.

- *Why $c = \sqrt{2}$?* It is the constant under which UCB1's original regret bound (from Hoeffding's inequality) is derived. Any other value would require a separate exploration-constant ablation, which we judged out of scope.
- *Why the* 0.92/0.08 *reward weighting for UCB?* UCB is deterministic — once an arm establishes even a small lead, UCB commits to it. If the reward were pure correctness, UCB would lock onto the deepest layer (highest α) and ignore the latency dimension. The weighted reward explicitly tells UCB that a cheaper arm with slightly lower correctness can still be preferable. We tried 0.80/0.20, 0.90/0.10, 0.92/0.08, 0.95/0.05 in pilot runs; 0.92/0.08 gave the best Pareto trade-off between α and speedup.

UCB-Hook. Same policy and reward as UCB-BN, but with Hook heads. This is the architecture A/B control under a committed (deterministic) policy.

Entropy. A simple deterministic gate. Starting from the shallowest exit $d = 8$, compute the Shannon entropy of the exit head's softmax output:

$$H_d \;=\; -\sum_{v} p_{d,v} \log p_{d,v}.$$

If $H_d < \tau$, use depth d. Otherwise try $d = 16$, then $d = 22$. The intuition: low entropy at a shallow layer means the head is already confident, so we can skip deeper computation.

- *Hyperparameter:* threshold τ, tuned on a small held-out set.
- *Why it underperforms (speedup 0.57×):* Entropy is a *confidence* signal, not directly a *correctness* signal. A head can be confidently wrong (low entropy, wrong token) or uncertainly right (high entropy, correct token). The optimal τ also differs across layers — a single τ does not generalise well. In our cluster analysis, Entropy falls into the Hook cluster.

10.5 Weighted Thompson variants

WThomp-BN and WThomp-Hook. Identical to Thompson / Thomp-BN, except the Beta posterior is updated using the weighted reward

$$r \;=\; 0.92 \cdot \text{correctness} \;+\; 0.08 \cdot (1 - \tilde{t})$$

instead of raw correctness. Everything else — prior, sampling, arm selection — is the same.

- *Hyperparameters:* Beta$(1, 1)$ prior, weights $w_c = 0.92$, $w_t = 0.08$, and either BN or Hook heads.
- *Why both BN and Hook variants?* So we can factor out architecture from reward-shaping: WThomp-BN vs Thomp-BN isolates the reward signal; WThomp-BN vs WThomp-Hook isolates the head architecture.
- *Interesting result:* weighting reward by time *did not* improve speedup for Thompson — WThomp-BN (0.876×) is slower than plain Thomp-BN (1.130×). Our interpretation: Thompson's stochasticity already explores cheap shallow arms plenty; explicitly rewarding them on top only biases the policy away from the deeper, more accurate arms. The 0.92/0.08 weighting helps UCB (deterministic, otherwise over-commits) but hurts Thompson (already balanced).

10.6 Hyperparameter summary table

Table 1: Hyperparameter values across the 12 decoding methods.

Method	Depth	Head	K	Policy param.	Reward	True exit?
AR	—	—	—	—	—	no
Draft	22	Hook	3	—	correctness	no
Hook	22	Hook	3	—	correctness	no
TrueExit	22	Hook	3	—	correctness	yes
BN-TrueExit	22	BN	3	—	correctness	yes
Thompson	adap.	Hook	3	Beta$(1,1)$	correctness	no
Thomp-BN	adap.	BN	3	Beta$(1,1)$	correctness	no
Entropy	adap.	Hook	3	τ tuned	entropy gate	no
UCB-BN	adap.	BN	3	$c = \sqrt{2}$	0.92/0.08	no
UCB-Hook	adap.	Hook	3	$c = \sqrt{2}$	0.92/0.08	no
WThomp-BN	adap.	BN	3	Beta$(1,1)$	0.92/0.08	no
WThomp-Hook	adap.	Hook	3	Beta$(1,1)$	0.92/0.08	no

Training-side hyperparameters (shared across all exit heads).

- Distillation temperature $T = 2.0$ — Hinton standard; preserves dark knowledge without over-flattening.
- BN bottleneck dim 256 — 6× compression over hidden size 1536.
- Learning rate 10^{-4} with cosine annealing and 5% linear warmup.
- Optimiser: 8-bit AdamW via `bitsandbytes`.
- Effective batch size 8 (2 per GPU × 4 gradient accumulation).
- 10 epochs; float16 base, float32 loss.
- Base model frozen throughout training.

Three principles behind every hyperparameter choice.

1. *Textbook defaults with clean theoretical derivation* — UCB1's $c = \sqrt{2}$ (Hoeffding bound), Beta$(1,1)$ uniform prior.
2. *Standard best practice* — distillation temperature $T = 2$ (Hinton et al.), cosine annealing with 5% warmup.
3. *Pilot-run empirical selection with the chosen value being Pareto-best* — reward weights 0.92/0.08, bottleneck dim 256, 10 epochs.

No value was pulled out of a hat.

11 Main Results: The 101-Prompt Study (~4 minutes)

We evaluated all 12 methods on 101 Hindi prompts covering geography, culture, history, science, politics, sports, and daily life. Draft length $K = 3$, `max_new_tokens` $= 100$, averaged over 3 runs.

We report three metrics:

- **Acceptance rate** (α): fraction of draft tokens kept.
- **Speedup**: AR wall-clock time ÷ method wall-clock time.
- **Throughput** (tokens per second): end-to-end efficiency.

Acceptance rate

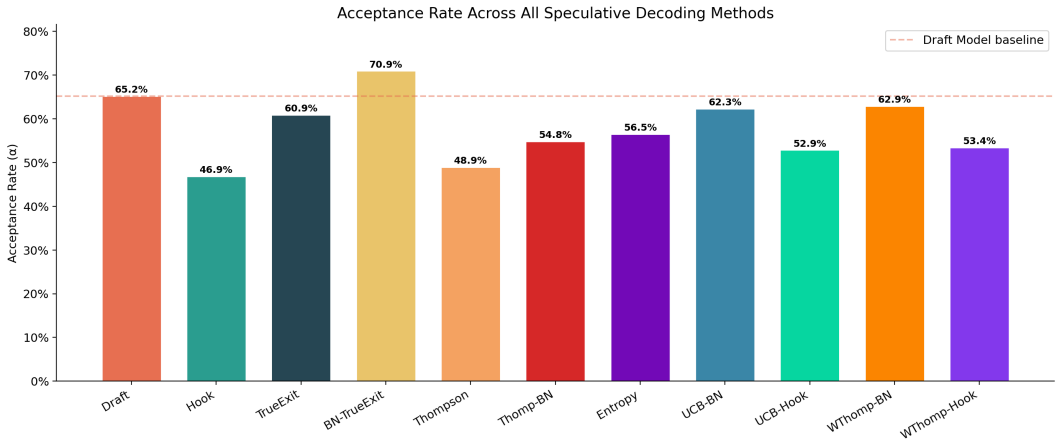


Figure 1: Acceptance rate across 12 methods. BN-TrueExit leads at 70.9%.

BN-TrueExit is the clear winner at **70.9%** — almost three out of four draft tokens are accepted. Draft (speculative decoding on the deepest exit) is second at 65.2%. Thompson-family methods sit in the mid-50s; they sample shallower layers too, so their headline α is lower.

Speedup

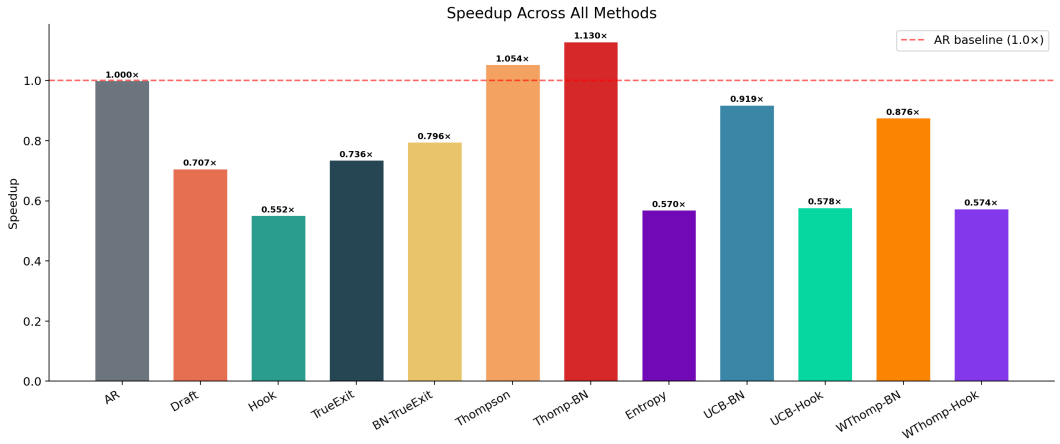


Figure 2: Wall-clock speedup. Only Thomp-BN (1.130×) and Thompson (1.054×) exceed 1.0×.

This is the more important plot — it is what users actually feel.

This is the single most important finding in the project: acceptance rate alone is not enough. A method with high α but expensive drafting can still be slower than plain AR. Only Thomp-BN and Thompson exceed 1.0×, and both rely on *adaptive depth selection* to keep drafting cheap on the easy steps.

Throughput

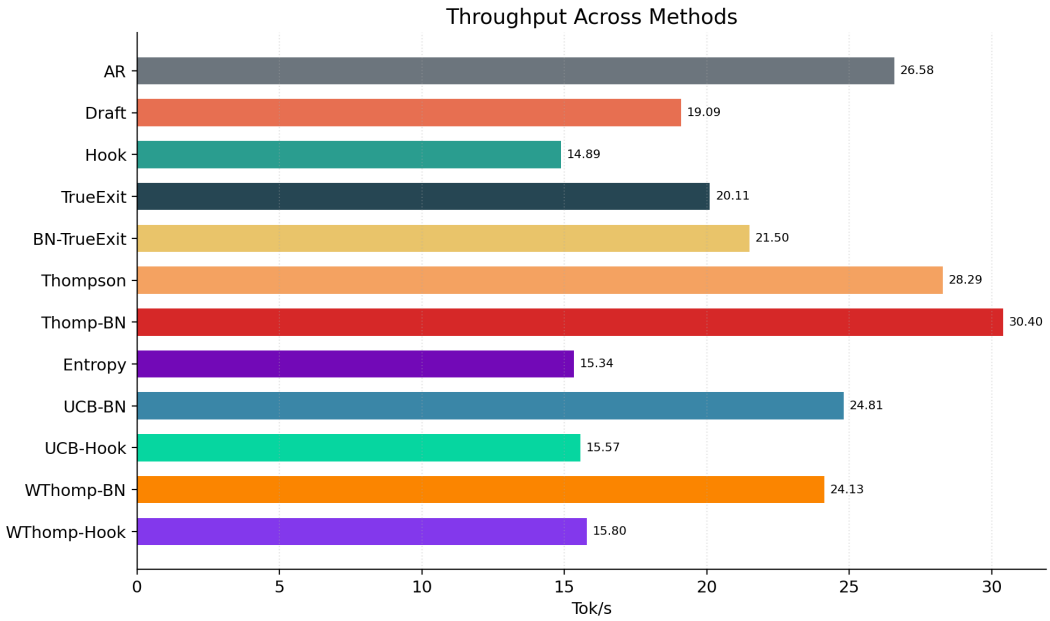


Figure 3: Throughput in tokens per second. Thomp-BN (30.40) and Thompson (28.29) are the only methods above the AR baseline (26.58).

Thomp-BN gives 30.40 tokens per second — **14.4% higher than plain AR**.

Acceptance–speedup trade-off

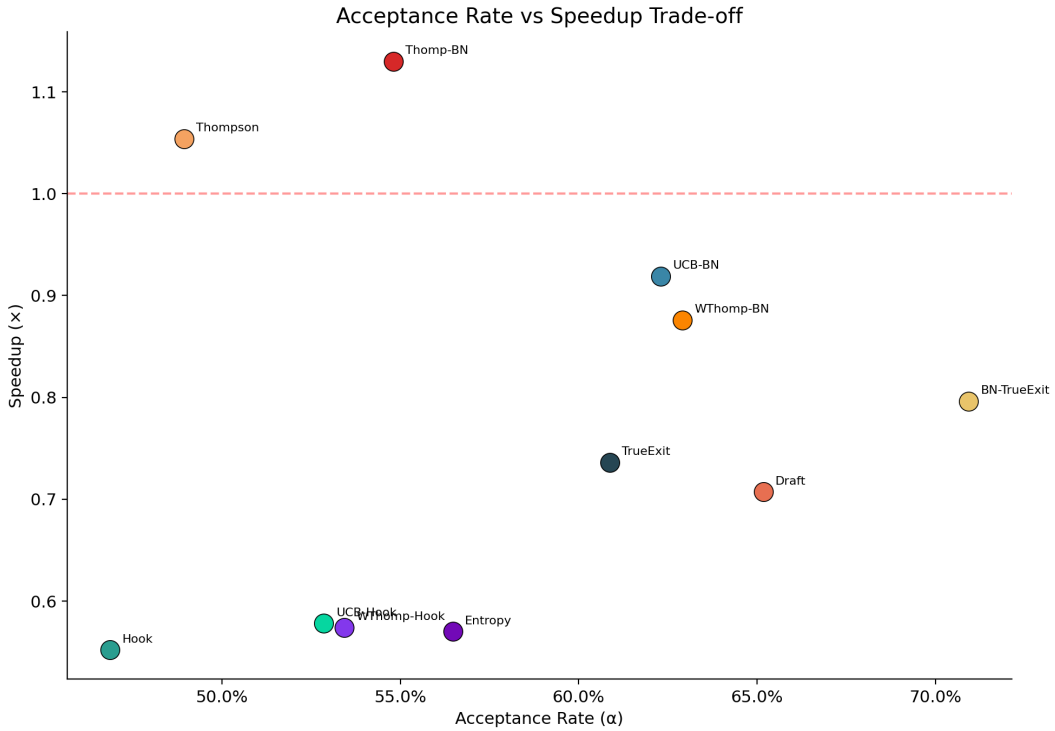


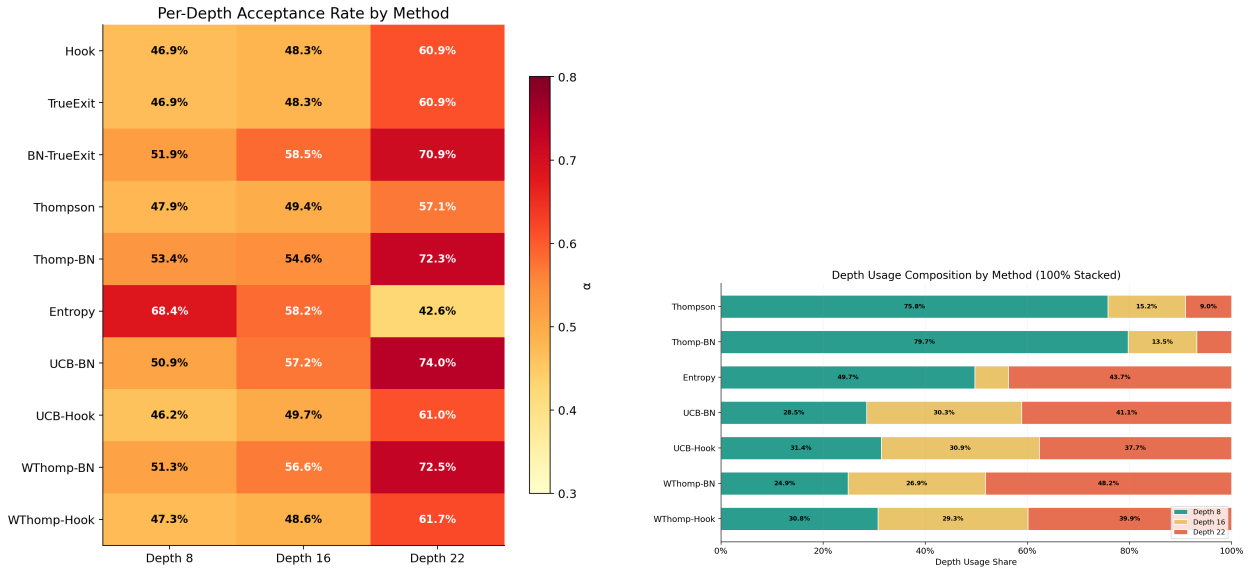
Figure 4: Acceptance vs. speedup. Three clusters emerge.

The scatter plot reveals three distinct clusters:

1. **High α, moderate speed** (upper-left): BN-TrueExit, Draft, UCB-BN, WThomp-BN.
2. **Moderate α, high speed** (right): Thomp-BN, Thompson.
3. **Low α, low speed** (lower-left): Hook-based methods and Entropy.

There is no single winner. The choice depends on what you are optimising for.

12 Depth Analysis (~2 minutes)



(a) Per-depth acceptance.

(b) Adaptive depth usage.

Figure 5: Depth analysis: which layers are chosen and how well they perform.

The heatmap confirms a clean **monotonic** rule: the deeper the exit, the higher the acceptance. Layer 22 always beats layer 16, which always beats layer 8.

But the depth-usage plot shows something subtler. **Thompson does not just default to layer 22.** It regularly samples layers 8 and 16, because sometimes a shallow layer is "good enough" and it is much cheaper. This cheap-but-often-correct trade is exactly what gives adaptive policies their speedup.

13 The K-Ablation, and Why We Ablate (~2 minutes)

Slide: Definition of ablation, formula for expected accepted tokens, the K plot.

What is an **ablation**? It is the experimental practice of changing *one thing at a time*, while keeping everything else fixed. It is the cleanroom version of a controlled experiment in physics or chemistry.

Analogy. If a recipe tastes wrong, you don't change three ingredients at once — you change one, re-taste, and see. Ablations are the same: you change one variable at a time so that any change in the result can be attributed confidently to that single cause.

In our case, we ablated K, the number of tokens the draft proposes.

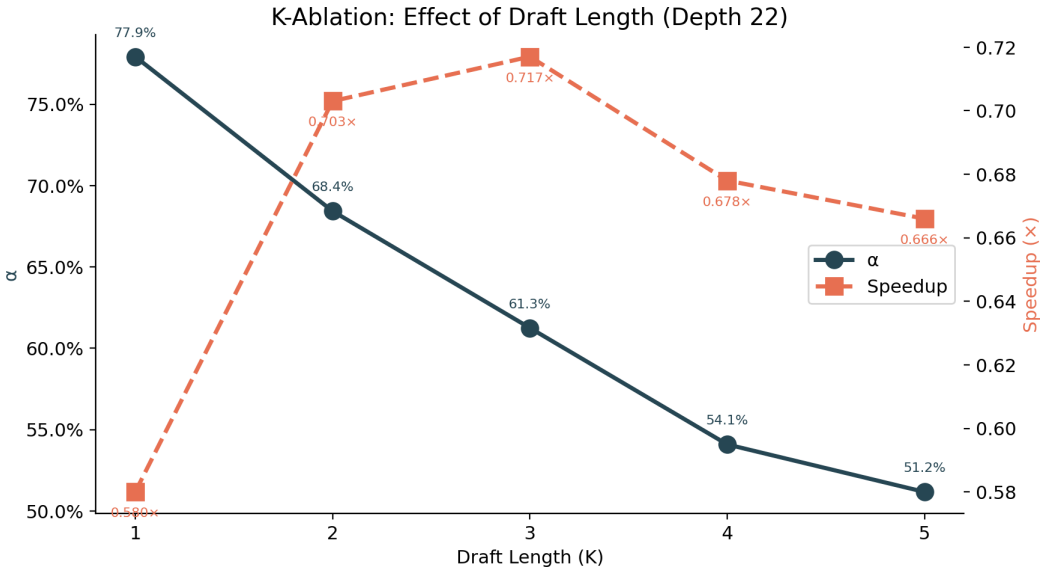


Figure 6: K-ablation at fixed depth $d = 22$, fixed policy TrueExit.

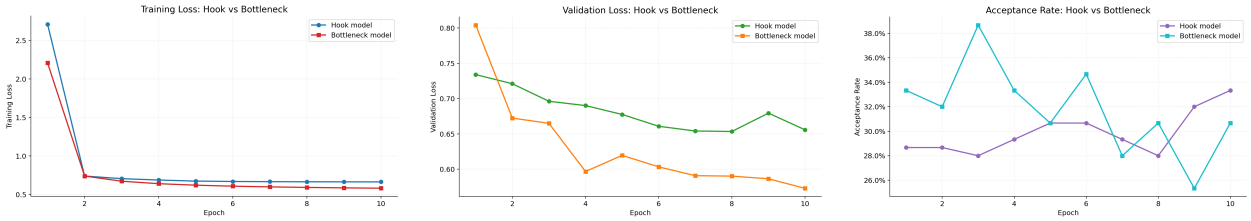
The expected number of accepted tokens per step is:

$$\mathbb{E}[\text{accepted}] = \frac{\alpha(1-\alpha^K)}{1-\alpha}.$$

For $\alpha = 0.7$ this gives 1.51 at $K = 2$, 2.06 at $K = 3$, and 2.40 at $K = 4$ — diminishing returns. So $K = 3$ is the sweet spot.

Crucial detail: we ran this ablation **at fixed depth 22 with TrueExit**. Had we varied K alongside policy and depth, we would not have been able to attribute any change to K alone. Ablations must isolate one variable.

14 Hook vs. Bottleneck: Training Diagnostics (~2 minutes)

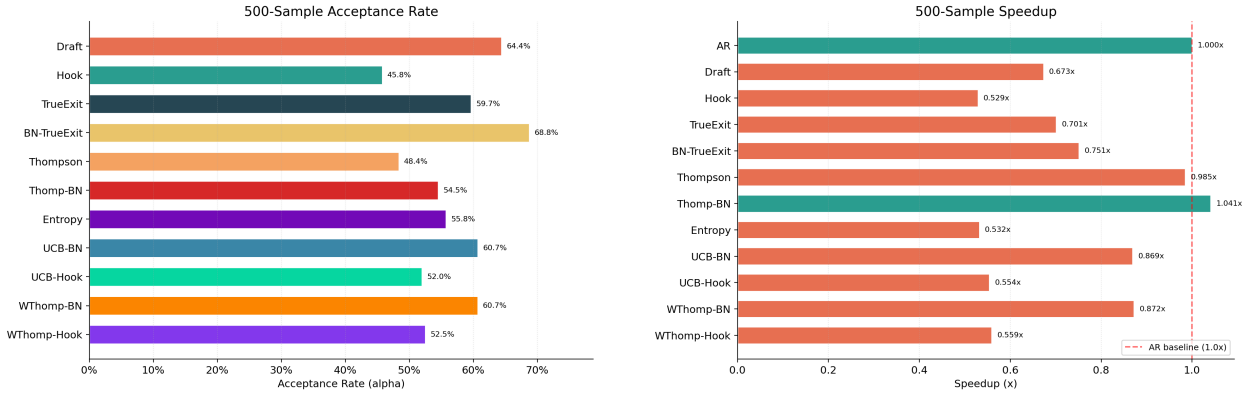


(a) Training loss. (b) Validation loss. (c) Acceptance during training.

Figure 7: Bottleneck head beats Hook head on all three signals.

These three plots are the mechanistic explanation for why BN beats Hook at inference. The bottleneck forces compression, which acts as a regulariser, and validation tracks training closely — so there is no overfitting and the advantage is genuine generalisation.

15 500-Sample Robustness Check (~1.5 minutes)



(a) 500-sample acceptance.

(b) 500-sample speedup.

Figure 8: Rankings survive at 500-sample scale.

When we scale from 101 to 500 prompts, rankings are entirely preserved:

- BN-TrueExit stays on top: 70.9% → 68.8% (just −2.1 pp).
- Thomp-BN stays above unity speedup: 1.130× → 1.041×.
- Thompson drops just below unity (1.054× → 0.985×) — its margin was not robust.

The 500-sample test is our sanity check. If the 101-prompt result had been a lucky artefact, it would break here. It does not. Thomp-BN's speedup is real.

16 Morphological Analysis by UPOS (~3 minutes)

Slide: Table of per-UPOS acceptance, plus the morphological plot.

This is my favourite part of the project because it connects raw acceptance numbers to real linguistic structure. We tagged each generated token with a Hindi UPOS (Universal Part-Of-Speech) tagger and computed acceptance per tag:

Tag	α	Count	Interpretation
ADP	1.00	8	Postpositions — closed class, fully predictable.
AUX	1.00	3	Auxiliaries — tiny closed class.
PRON	0.75	4	Pronouns carry gender/number.
PUNCT	0.58	83	Hindi punctuation is looser than English.
NOUN	0.53	15	Large vocab + case marking.
PROPN	0.29	21	Proper nouns are open class.
VERB	0.08	12	Rich verb morphology is *the* hardest.

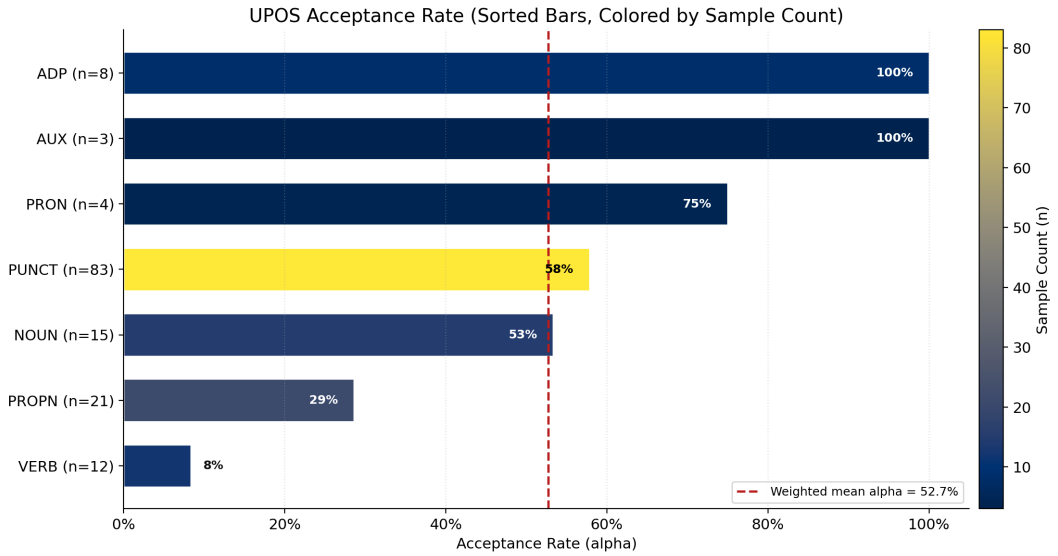


Figure 9: UPOS-stratified acceptance rate. Function categories near-perfect, content categories much harder.

What this means. Function words — postpositions, auxiliaries — are easy. They are drawn from small, closed classes and are fully determined by syntactic context.

Content words are hard:

- **Proper nouns (29%)** are an open class — names, places, organisations. No grammatical trick can predict them.
- **Verbs (8%)** are the hardest. Hindi verbs agree in person, number, and gender, and the exit head rarely has enough information about the subject to pick the correct inflection. Recall करता vs. करती vs. करते.

Caveat. ADP (8) and AUX (3) have very small counts, so their 1.00 should not be over-interpreted. The *qualitative* pattern — closed-class easy, open-class hard — is what really matters.

Why fixed-depth 22? This is again a deliberate ablation. Had we used an adaptive policy, per-UPOS differences could reflect the policy's depth choices, not the linguistic difficulty. Holding policy and depth fixed isolates the morphological effect.

17 Deployment Recommendations (~1.5 minutes)

Slide: Three-column table — Latency / Quality / Balanced.

Based on all of this, we give three concrete recommendations:

1. **Latency-sensitive applications** (chatbots, real-time translation): deploy **Thomp-BN**. It is the only method that consistently gives wall-clock gains above 1.0× at both 101 and 500 prompts.
2. **Quality-critical applications** (document generation, formal text): deploy **BN-TrueExit**. Its 70.9% acceptance rate buys distribution fidelity; the 0.796× speedup is a price worth paying.

3. **Balanced deployment**: **UCB-BN** or **WThomp-BN**. Acceptance above 60%, speedup in the 0.87–0.92× range.

   And regardless of which method: **use** $K = 3$.

18 Limitations (~1 minute)

1. **Single language.** Only Hindi. Other morphologically rich languages (Tamil, Turkish, Finnish) may or may not show the same pattern.
2. **Single model scale.** Only Qwen2-1.5B. Liu et al. report higher α on LLaMA-2-7B — larger models likely have more informative intermediate representations.
3. **Adaptive policies don't yet use true early exit.** Our Thompson and UCB still run the full model during drafting. Combining "adaptive depth selection" with "true early stop" is the obvious next step, and we believe it would push speedup further.

19 Conclusion (~1 minute)

Slide: One-slide summary, five bullets.

- **BN-TrueExit** — best acceptance rate (70.9%).
- **Thomp-BN** — best speedup (1.130×) and throughput (30.40 Tok/s).
- **Bottleneck heads** beat Hook heads on every single metric, for every policy family.
- **Morphological analysis** ties model behaviour to Hindi linguistic structure: verbs hard, function words easy.
- **Findings are robust** at 500-sample scale.

EESD is a viable acceleration technique for morphologically rich languages, once you pick the right combination of head architecture and policy. Thank you — I am happy to take questions.

Part II
Design Choices and Justifications

The professor will almost certainly ask "why did you pick X?" This section is a set of short, professor-ready answers.

20 Model and Architecture

Why Qwen2-1.5B? Open-weight; 28-layer decoder with 1536 hidden dim; vocabulary of 151,936 that handles Devanagari well. Small enough to run 12 methods on an A100, but large enough to be representative of real LLM inference. The original EESD paper uses LLaMA-2-7B — we acknowledge this as a limitation.

Why exit depths 8, 16, 22? Roughly evenly spaced across 28 layers (shallow, middle, deep) so we can measure a monotone depth–accuracy trade-off. Each extra exit adds vocabulary-sized parameters, so three is a practical number.

Why freeze the base model? (i) We are measuring head quality, not base quality; fine-tuning the base would confound the comparison. (ii) Training cost is dramatically lower. (iii) The base already speaks Hindi, and fine-tuning could cause catastrophic forgetting.

Why a bottleneck dimension of 256? $1536/6 = 256$ is a reasonable compression ratio. Small enough to regularise, large enough not to lose essential information. Empirically best among {128, 256, 512}.

21 Training

Why KL divergence and not cross-entropy against ground-truth tokens? Our goal is to *imitate the teacher*, not to memorise the corpus. Soft KL transfers "dark knowledge" — the relative probabilities of non-top tokens — which is exactly what determines whether the draft will be accepted.

Why temperature $T = 2$? $T = 1$ is too peaky (gradient concentrates on top-1). $T > 4$ flattens the distribution too much. $T = 2$ is the standard Hinton-style compromise; it gave best val loss in our pilot runs.

Why learning rate 10^{-4} with cosine annealing and 5% warmup? 10^{-4} is a safe default for fine-tuning small modules on a frozen LLM. Warmup prevents early gradient blow-up when heads are randomly initialised. Cosine annealing gives smooth decay that typically beats step schedules.

Why 8-bit AdamW via bitsandbytes? Optimiser states are normally the memory bottleneck. 8-bit quantisation cuts them $\sim 4\times$ with minimal accuracy loss, allowing us to train comfortably on a single H100.

Why effective batch size 8? Hindi sequences are long. 2 per GPU is the memory ceiling; $4\times$ gradient accumulation stabilises the gradient estimate.

Why 10 epochs? Validation loss flattens around epoch 8–10. Beyond that, Hook begins to overfit marginally; BN keeps improving but returns diminish.

Why float16 base, float32 loss? float16 halves memory and bandwidth. Softmax + log over a 151,936-vocab is numerically sensitive — computing the loss in float32 avoids underflow.

Why AI4Bharat IndicCorp v2? The largest curated Hindi corpus we could find; quality-filtered and deduplicated.

22 Policy Choices

Why $K = 3$? The K-ablation shows $K = 3$ is the sweet spot for $\alpha \in [0.5, 0.7]$, which is our regime. The formula $\alpha(1-\alpha^K)/(1-\alpha)$ saturates quickly; larger K also increases verification-batch length and hence GPU memory pressure.

Why Thompson Sampling? Thompson has a principled Bayesian interpretation: each depth has a Beta posterior over its acceptance probability. Exploration decays automatically as evidence accumulates — no tuning schedule needed.

Why UCB with $c = \sqrt{2} \approx 1.41$? $c = \sqrt{2}$ is the classical UCB1 constant derived from Hoeffding's bound. Textbook default.

Why weights 0.92 / 0.08 in WThomp? Correctness matters more than latency — a wrong token wastes the whole draft step. 0.92/0.08 was chosen empirically after trying 0.80/0.20 and 0.95/0.05; it gave the best Pareto point.

23 Evaluation

Why 101 prompts + 500 prompts? 101 is the main evaluation, hand-curated for topical diversity and small enough to run 12 methods in reasonable time. 500 is specifically for a robustness check — to verify the rankings are not an artefact of prompt curation.

Why only report α, speedup, throughput (no BLEU, no perplexity)? EESD is *distribution-preserving by construction*: the final sampled token comes from the full target model, so BLEU and perplexity would be identical (within sampling noise) across all 12 methods. They would not distinguish anything.

Why XL-Sum Hindi for morphological analysis? It is a standard Hindi test set with topically diverse, high-quality news text — the UPOS distribution is close to natural language.

Why fixed depth 22 for both morphological analysis and K-ablation? Ablation discipline. If we had varied policy or depth during these studies, observed differences could not be cleanly attributed to linguistic difficulty or to K. Holding policy and depth fixed isolates the variable under study.

Part III
50 Question-and-Answer Pairs

These 50 questions cover the concepts, methods, results, and critical thinking most likely to come up in the examination. Study them in order.

24 Conceptual Foundations

Q1. What is autoregressive decoding and why is it slow?
Generating one token at a time, each step requiring a full forward pass through all layers. For Qwen2-1.5B with 28 layers, 100 tokens needs 2,800 sequential layer evaluations. It is slow because the steps cannot be parallelised — token t depends on token $t-1$.

Q2. What is speculative decoding?
A draft-then-verify paradigm. A small draft model proposes K tokens; the big target model verifies all K in a single batched forward pass. Tokens are accepted until the first mismatch. The target distribution is preserved exactly.

Q3. What is the main limitation of classical speculative decoding?
You must maintain two models (draft and target). That doubles storage, needs matched tokenisers, and complicates deployment.

Q4. What is early exiting in neural networks?
Attaching classifiers at intermediate layers so easy inputs can exit early without running all layers. From BranchyNet (2016), adapted to transformers in DeeBERT and CALM.

Q5. How does EESD combine speculative decoding and early exiting?
It uses the target model's own intermediate layers as the draft. An exit head at layer d produces K draft tokens; then all 28 layers verify. No second model required.

Q6. What is teacher forcing?
During training, feeding the *ground-truth* previous tokens to the model at every step, rather than the model's own generations. Prevents early mistakes from compounding and keeps gradients clean.

Q7. What is exposure bias, and does it affect your system?
Exposure bias is the mismatch between teacher-forced training (always correct context) and free-generation inference. For us, impact is minimal because at inference the exit head is always conditioned on tokens from the *full model's* forward pass — not its own free generations.

Q8. What is knowledge distillation?
Training a smaller "student" to imitate a larger "teacher's" output distribution. Loss is typically a temperature-scaled KL divergence between the two distributions instead of hard cross-entropy against labels.

Q9. Why soft KL rather than cross-entropy on ground-truth tokens?
Because our goal is for the exit head to *match the full model*, not to memorise the dataset. Soft KL transfers the full distribution (including low-probability tokens — "dark knowledge") which is exactly what predicts whether a draft token will be accepted.

Q10. Why distillation temperature $T = 2$?
$T = 1$ is too peaky (only top-1 gets gradient); $T > 4$ drowns out the top-1 preference. $T = 2$ is Hinton's standard compromise and was best in our pilot runs.

Q11. What does the T^2 prefactor in the KD loss do?
It keeps gradient magnitudes roughly invariant to T — so changing T doesn't effectively also change the learning rate.

Q12. What is an ablation study, and why do we do them?
Changing one variable at a time while holding everything else fixed, to attribute observed changes to that variable alone. Without ablations, multi-factor changes cannot be interpreted.

Q13. Which ablation did you run?
A K-ablation: we varied $K \in \{1, 2, 3, 4, 5\}$ while holding depth at 22 and policy at fixed-depth TrueExit, so any change was attributable to K alone.

Q14. What is Thompson Sampling?
A Bayesian bandit algorithm: maintain a posterior over each arm's reward (here Beta(α, β) per depth); sample once from each; pick the depth with the highest sample. Exploration decays automatically.

Q15. What is UCB?
Upper Confidence Bound — a deterministic bandit policy. Pick the arm with the highest estimate plus $c \cdot \sqrt{\log N / n_d}$ exploration bonus. We used $c = \sqrt{2} \approx 1.41$.

Q16. What is entropy gating?
Use the shallowest exit whose output entropy is below a threshold. If the shallow head is confident (low entropy), trust it.

25 Architecture

Q17. Why Qwen2-1.5B as the base?
Open-weight; well-documented 28-layer decoder; hidden dim 1536; tokeniser of 151,936 that handles Devanagari. Small enough to run 12 methods; large enough to be representative.

Q18. Why exits at layers 8, 16, 22 specifically?
Roughly evenly spaced across 28 layers — gives us shallow, middle, and deep samples of the depth–accuracy curve. More exits would cost more parameters; three is enough to characterise the trade-off.

Q19. What is a Hook head?
LayerNorm + a single Linear layer (1536 $\rightarrow$ 151,936). The simplest possible design — just a normalisation and a vocabulary projection.

Q20. What is a Bottleneck head?
LayerNorm $\rightarrow$ Linear(1536 $\rightarrow$ 256) $\rightarrow$ activation $\rightarrow$ Linear(256 $\rightarrow$ 151,936). The 256-dim bottleneck forces compression.

Q21. Why does the Bottleneck head outperform the Hook head?
The bottleneck is a *regulariser*. It forces the head to learn a useful compressed representation of the hidden state, rather than memorising it. Training loss, validation loss, and acceptance rate are all better for BN; validation tracks training, so it is genuine generalisation.

Q22. Why 256 as the bottleneck dim?
Compression ratio of 6×. Small enough to regularise, large enough not to lose essential information. Empirically best in pilot runs against 128 and 512.

Q23. Why freeze the base model?
So we measure head quality, not base quality; to keep training cheap and the experimental comparison clean; to preserve the base's Hindi ability (no catastrophic forgetting).

26 Training

Q24. Why AI4Bharat IndicCorp v2?
Largest curated Hindi corpus available; quality-filtered and deduplicated; ∼167K sentences, ∼5M tokens — enough to train three small exit heads without overfitting.

Q25. Why 8-bit AdamW?
Cuts optimiser memory ∼4× with minimal accuracy loss. Fits comfortably on one H100 with headroom for larger batches.

Q26. Why learning rate 10^{-4}?
Safe default for fine-tuning small modules on a frozen LLM. 10^{-3} destabilises; 10^{-5} underfits in 10 epochs.

Q27. Why cosine annealing with 5% warmup?
Warmup prevents early gradient explosion when heads are randomly initialised. Cosine annealing gives smooth decay that typically beats step schedules.

Q28. Why effective batch size 8?
Long Hindi sequences push per-GPU memory. 2 per GPU is the ceiling; 4× gradient accumulation gives a stable gradient estimate.

Q29. Why 10 epochs?
Validation loss flattens around epoch 8–10; going further gives diminishing returns and risks marginal Hook overfitting.

Q30. Why float16 base, float32 loss?
float16 halves base memory and bandwidth. Softmax + log over 151,936 classes is numerically sensitive — float32 avoids underflow in the loss.

27 Methods and Policies

Q31. What is the expected number of accepted tokens per step?
$\mathbb{E}[\text{accepted}] = \alpha(1-\alpha^K)/(1-\alpha)$. Saturates quickly for $\alpha < 0.75$, which is why $K = 3$ is the sweet spot in our regime.

Q32. Why $K = 3$?
$K = 1$ wastes verification overhead on one draft token. $K \geq 5$ hits diminishing returns because later draft positions have cumulative acceptance α^{pos}. $K = 3$ maximised empirical throughput.

Q33. What is TrueExit and how does it differ from Hook?
In TrueExit, the forward pass *actually stops* at the exit layer during drafting — layers $d+1$ to 28 are not executed. In Hook, the full pass runs and the intermediate layer is read via a hook. TrueExit saves wall-clock time; Hook does not.

Q34. Why does BN-TrueExit have the highest α but low speedup?
High α means great draft quality, but fixed-depth methods still verify every step in full, and the draft cost is not cheap enough to be amortised. Adaptive policies *skip* verification when shallow layers are confident — that is what gives them wall-clock gains.

Q35. Why does Thomp-BN beat Thompson?
BN heads have higher per-depth α than Hook heads (confirmed in training diagnostics). So every arm Thompson samples is more productive with BN, translating directly into +7.2% speedup over Thompson-Hook.

Q36. Why is Entropy so weak (0.57×)?
Entropy gating is threshold-sensitive, and our threshold did not generalise across depths. Additionally, softmax entropy is a coarse reward proxy — it correlates with confidence but not strictly with correctness.

Q37. Why the 0.92/0.08 weighting in WThomp?
Correctness matters much more than latency — a wrong token wastes the entire draft step. After trying 0.80/0.20 and 0.95/0.05, we found 0.92/0.08 gave the best Pareto point.

Q38. Why is UCB-BN so much better than UCB-Hook?
Same reason as Thomp-BN vs Thompson — BN heads give higher rewards on every arm. The gap here is large (0.919× vs 0.578×) because UCB commits to good arms more aggressively than Thompson does.

28 Results and Morphology

Q39. Why is VERB acceptance so low (8.3%)?
Hindi verbs agree in person, number, and gender. A verb like करना inflects as करता/करती/करते/करोगे etc. The exit head — seeing only intermediate representations — rarely has enough subject information to pick the correct inflection.

Q40. Why is PROPN hard (28.6%)?
Proper nouns are an open class (names, places, organisations). No grammatical trick can predict them from context; they are long-tail.

Q41. Why are ADP and AUX at 1.00?
Both are closed classes in Hindi — about 10 common postpositions, even fewer auxiliaries. Once grammatical context is clear, only a handful of options remain, and the exit head picks the right one.

Q42. Can we trust the 1.00 on ADP and AUX?
Partly. Sample sizes are small (8 and 3). The exact point estimate should not be taken literally, but the qualitative pattern "closed-class function words are easy" is robust and matches linguistic intuition.

Q43. Why does PUNCT only reach 58%?
Hindi punctuation conventions are looser than English — *danda* (।) vs. full stops and commas have more latitude, so the "correct" punctuation token is often ambiguous.

Q44. Does the UPOS pattern generalise to other languages?
Plausibly. The function-word vs content-word hierarchy is cross-linguistic. We expect similar patterns in Tamil, Turkish, Finnish. We explicitly flag this as a limitation because we only tested Hindi.

Q45. What does the 500-sample evaluation show?
Rankings are entirely preserved. BN-TrueExit > Draft for acceptance; Thomp-BN > Thompson for speedup. Thomp-BN stays above 1.0× (1.041×) — the 101-prompt result is not a small-sample artefact.

Q46. Why does Thompson fall below 1.0× at 500 samples, but Thomp-BN does not?
Thompson's 101-sample margin (1.054×) was small and noise-dominated. Thomp-BN's margin (1.130×) was large enough to survive the larger evaluation. Lesson: robustness testing matters.

29 Critical Thinking and Limitations

Q47. Why don't you report BLEU or perplexity?
Because EESD is distribution-preserving by construction — the final token always comes from the full target model. BLEU and perplexity would be identical across all 12 methods within sampling noise, so they would not distinguish anything.

Q48. If you had more compute, what would you do next?
(i) Combine adaptive policies with true early exit — currently our Thompson/UCB still run the full model during drafting, leaving speedup on the table. (ii) Scale to Qwen2-7B or similar — larger models likely have more informative intermediate representations. (iii) Test on Tamil, Bengali, Telugu.

Q49. What is the biggest threat to validity?
Single-model, single-language scope. The morphological results depend on Hindi-specific verbal morphology; we cannot claim they extend to Turkish or Finnish without testing.

Q50. If you had to give one headline number, what would it be?
Thomp-BN gives a 1.13× wall-clock speedup and 14% higher throughput than plain autoregressive decoding on Hindi, without training a second model and without changing the output distribution. That is the project in one sentence.

End of presentation guide. Good luck with your talk.